

同濟大學

TONGJI UNIVERSITY

毕业设计（论文）

课题名称 面向混合检索的统一图索引构建

副标题 支持动态权重的稠密-稀疏向量检索方法

学 院 计算机科学与技术

专 业 计算机科学与技术

学生姓名 陈柯宇

学 号 2251318

指导教师 程鹏

日 期 2026.6

同济大学本科毕业设计（论文）信息说明页

课题名称：面向混合检索的图索引构建

成果类型： ☐ 毕业设计 ☒ 毕业论文 （勾选）

学科专业： 计算机科学与技术

内容简述 （请用 300 字以内简要概述：）

本文针对稠密向量与稀疏向量混合检索的场景，提出了一种支持动态混合权重的统一近邻图检索方法。现有主流混合检索方案普遍采用“稠密-稀疏两路召回+全量重排”的架构，该架构不仅需要维护两套独立索引，系统复杂度与存储成本高，且在中间权重场景下存在召回下降问题，导致召回率不稳定。针对上述不足，本文提出了一种新的混合图索引架构，合并了各个混合权重下的 KNN 子图，实现“单索引、多语义”的检索能力，且无需重建索引即可高效支持任意混合权重的查询。本文基于 MSMARCO 数据集进行实验，与传统方案相比，本方法在召回率和检索速率两项核心指标上均具有明显提升，有效验证了本方法在混合检索任务中的实用价值。

毕业论文：正文总字数 23014 字。

随附资料：

1. 程序代码

（请列出所有提交的支撑材料名称，如：毕业设计-全套图纸、毕业作品、计算书、程序代码、附录等）

面向混合检索的统一图索引构建

摘要

近年来，伴随着大语言模型的飞速发展，作为大模型检索增强生成系统（RAG）的关键技术，向量数据库和向量检索逐渐兴起。为了同时满足对文本语义的理解与对特定关键词的精确匹配，稠密向量与稀疏向量的混合检索技术受到了学术界与工业界的广泛关注。当前主流的混合检索方案通常采用“稠密-稀疏两路召回”的架构，但该方案在两路各自召回时容易丢失正确结果，导致召回率不稳定，并且两路召回的检索效率较低，此外该方案还需要维护两套独立的索引架构，导致系统复杂度上升。

针对上述挑战，本文提出并实现了一种面向混合检索的统一混合近邻图索引，将多阶段检索精简为单一图检索，与传统的两路召回等方案相比，能够更高效、高召回、自适应地支持任意动态混合权重的查询需求，证明了该方案的工程落地价值与广阔的应用前景。

关键词：向量检索，混合检索，图索引，近似最近邻搜索

Unified Graph Index Construction for Hybrid Retrieval

ABSTRACT

In recent years, with the rapid development of large language models, vector databases and vector retrieval, as key technologies of Retrieval Augmented Generation(RAG) systems, have gradually emerged. In order to both satisfy the understanding of text semantics and the accurate matching of specific keywords at the same time, the hybrid retrieval technology of dense vector and sparse vector has been widely concerned by academia and industry. The current mainstream hybrid retrieval method usually adopts the architecture of "dense-sparse two-way recall", but this method is easy to lose the correct results when dense vector and sparse vector are retrieved separately, resulting in unstable recall rate, and the retrieval efficiency of two-way recall is low. In addition, this method also needs to maintain two sets of independent index structures, which leads to the increase of system complexity.

In this paper, we propose a unified hybrid nearest neighbor graph index for hybrid retrieval, which simplifies multi-stage retrieval into a single graph retrieval. Compared with the traditional two-way recall method, it can adaptively support the query requirements of any dynamic hybrid weight more efficiently, with high recall, and proves the engineering value and broad application prospect of our method.

Key words: Vector Retrieval, Hybrid Search, Graph Index, Approximate Nearest Neighbor Search

目 录

1	引 言	1
1.1	向量检索研究背景与意义	1
1.2	向量检索常用的方法	2
1.2.1	基于空间划分的方法	2
1.2.2	基于哈希的方法	4
1.2.3	基于量化的方法	5
1.2.4	基于图的方法	8
1.3	混合检索的基本概况	13
1.4	本文所做的工作	16
2	混合检索与统一图索引方法	19
2.1	数据的向量表征	19
2.2	混合距离度量	20
2.3	Baseline 架构分析	21
2.3.1	Baseline 的耗时分析	21
2.3.2	Baseline 的召回分析	21
2.4	混合图算法设计	22
2.4.1	混合图算法设计的数学直觉	22
2.4.2	统一混合近邻图的建图算法	23
2.4.3	统一混合近邻图的搜索算法	25
3	实验设计与评估方法	28
3.1	实验环境与数据集	28
3.1.1	实验环境	28
3.1.2	数据集处理	28
3.2	评价指标与对比基线	29

3.2.1 评价指标	29
3.2.2 baseline 方案设定	29
4 实验结果与性能分析	31
4.1 实验结果	31
4.2 性能分析与讨论	32
5 结论和展望	34
5.1 研究结论	34
5.2 未来工作展望	35
参考文献	36
谢辞	39

1 引言

1.1 向量检索研究背景与意义

在如今信息爆炸的时代，人类社会产生的数据量正以前所未有的速度不停增长，伴随着互联网、云计算还有物联网技术的快速发展，各种信息系统每天都会生成海量的数据。根据相关研究统计，目前全球的数据总规模已经达到了 Zettabyte（ZB）级别，而且还在保持指数级增长的趋势，这些数据里，有很大一部分都不是传统数据库里那种结构清楚的表格数据，而是以文本、图像、音频、视频等不同形式存在的非结构化数据，这类数据一般都没有统一的结构标准，传统关系数据库里的索引技术，很难直接用在这些数据上。

所以，如何从海量非结构化数据里，快速又准确地找出用户需要的信息，已经是现代计算机科学里一个重要的研究方向。过去的信息检索技术大多靠关键词匹配来运行，像是倒排索引、TF-IDF 这些方法都属于这一类，这些办法处理短文本、或者关键词清晰的查询时效果尚可，但碰到复杂语义、长文本上下文还有跨模态数据的时候，它的检索能力就常常不够用了。

近年来，在深度学习技术快速发展的背景下，表示学习（Representation Learning）方法慢慢变成解决这类问题的重要办法。利用深度神经网络模型，可以把原本复杂多样的非结构化数据，转换成对应的高维向量，这类向量一般被叫做嵌入向量（Embedding），在向量空间里，语义相近的数据会被映射到距离比较近的位置，语义差别大的数据则会分布在更远的地方。

这种向量化的表示方法，大幅简化了相似度计算的相关问题，原本没法直接量化的语义关系，现在能够借助数学距离的度量方式来描述。比如，在文本语义检索的任务里，系统只需要算出查询向量和数据库里各个向量之间的距离，比如常用的欧氏距离或是余弦相似度，这样就能找到语义最接近的结果，所以现在向量检索技术，慢慢变成了新一代搜索引擎、推荐系统还有各类人工智能应用里，非常关键的基础功能支撑。

向量检索最核心的问题可以简单概括成最近邻搜索（Nearest Neighbor Search, NNS），也就是在拿到给定的查询向量之后，从数据库里找出和它距离最近的那几个

向量。放在推荐系统里，这项技术能用来找和用户兴趣匹配度最高的商品或者内容；放到语义搜索系统里，就可以按照用户输入的问题，找到和问题语义最贴合的文档，到了智能问答系统里，它又能帮系统快速找到知识库里面相关的内容。

伴随着大语言模型（Large Language Models, LLM）逐步发展起来，向量检索技术的重要性得到了进一步提高。现在主流的大模型应用基本都会用到检索增强生成（Retrieval-Augmented Generation, RAG）框架。在这个框架里，系统会先利用向量检索从外部知识库中取出相关的文档，之后再依靠语言模型生成对应的回答，所以，向量检索模块其实可以看作大模型系统的“外部记忆”，它的性能好坏会直接影响最终生成回答的质量。

但是，放到实际场景里用的话，向量检索会遇到不少问题。现在深度学习模型生成的向量，维度一般都有几百甚至上千，而数据库里的数据量可能达到数亿甚至几十亿级别，这种情况下，要是还用传统的精确最近邻搜索方法，它的时间复杂度是 $O(ND)$ ，也就是得给所有向量挨个计算距离，放在大规模系统里肯定没法正常运行，而且在高维空间里还会出现大家都知道的“维度灾难”，就是随着维度变高，不同数据点之间的距离分布会慢慢变得均匀，这就会让传统空间索引结构的剪枝能力下降很多。

为了处理这个问题，研究者提出了近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）技术。这项技术允许在召回率上做一点点让步，以此换来查询速度的明显提高，在实际应用的系统里，这样的取舍通常是大家可以接受的，所以 ANNS 现在已经变成了向量检索系统里的主流技术方向。

1.2 向量检索常用的方法

针对高维向量的近似最近邻检索问题，学术界和工业界过去十几年里做了很多深入研究，也出现了不少经典的算法框架。按照底层数据结构和空间划分逻辑的区别，常用的向量检索方法大概可以分成四大类。

1.2.1 基于空间划分的方法

基于空间划分的方法是最早被用来做多维数据搜索的技术之一。这类方法的主要思路，是通过划分数据空间，把数据点整理成分层结构，这样在查询的时候，就能快速排除掉很多不相关的区域。

（1）KD-Tree

KD-Tree (K-Dimensional Tree) 是一种很经典的多维空间划分结构。它的基本思路，就是顺着坐标轴方向对数据空间做递归划分，搭建 KD-Tree 的时候，一般会先选某一个维度当作划分维度，再根据这个维度上数据点的中位数，把所有数据分成左右两个子集，之后再递归搭建出子树。

在查询步骤里，系统会先按照查询向量的坐标找到对应的叶节点，之后再往回搜索，看看其他分支里有没有距离更近的邻居。用这种方式处理，不用把整个数据集都扫一遍，就能提高搜索的效率。

KD-Tree 在低维数据（如 10 维以内）里的表现还不错，它的平均查询复杂度大概是 $O(\log N)$ 。但是，当数据维度变高之后，它的空间划分效果会快速变差，得访问非常多的节点，最终性能就跟线性扫描差不多了。

（2）Ball Tree

Ball Tree 是另外一种常用的空间划分结构。跟 KD-Tree 不一样，Ball Tree 用的是超球体 (Hypersphere)，不用超平面来划分空间，它的每一个节点都会包含一块球形区域，这块区域是由中心点还有半径来定义的。

实际查询的时候，如果查询点和某个球体之间的最小距离，已经比当前算出的最优距离要大，就可以直接剪掉这个节点，这样就能缩小要搜索的范围。

Ball Tree 在部分高维数据集上的表现会比 KD-Tree 更稳定，不过当维度特别高的时候，它还是会受到“维度灾难”的影响。

（3）倒排文件索引 (IVF)

为了适配更大规模的数据集，研究者提出围绕聚类思路设计的空间划分方法，其中最具代表性的就是倒排文件索引 (Inverted File Index, IVF)。

倒排文件 IVF 的基本思路是，先用 K-Means 算法把整个数据集分成若干个聚类中心，每个聚类中心会对应一个存储桶 (Bucket)。任意一个向量都会被分到，距离它最近的那个聚类中心对应的桶里。

在进行查询操作的时候，系统只需要访问和查询向量最接近的几个聚类桶就行，不用把整个数据集都扫一遍。这样做能明显缩小需要筛选的候选集范围。

IVF 方法现在在大规模向量检索系统里用得很广，尤其是和量化方法搭配使用的

时候（比如 IVFPQ），能在运行效率和存储空间占用之间找到比较好的平衡点。

但是，围绕空间划分做的这类方法，普遍都有一个共同的问题：如果数据本身维度比较高，空间划分的效果就会快速变差，进而让搜索的效率明显变低。所以，在现在的高维向量检索系统里，这类方法一般都得和其他技术搭配在一起使用。

1.2.2 基于哈希的方法

利用哈希的方法，是把高维向量映射到低维的离散空间，以此加快相似度搜索的速度。这种方法的主要思路是设计合适的哈希函数，让本身相似的向量，在哈希空间里依然有很大概率落到同一个分组里。

（1）局部敏感哈希（LSH）

局部敏感哈希（Locality-Sensitive Hashing, LSH）是这个领域里最有代表性的算法。它的基本原理是做出一组随机投影函数，把高维向量转换成二进制哈希值，举个例子，计算余弦相似度的时候，可以用随机超平面来做投影，之后按照向量落在超平面的哪一边，生成对应的二进制编码。

把多个哈希函数组合起来，能够搭建出多个哈希表结构。查询的时候，系统只需要打开跟查询向量哈希值一样或者接近的存储桶，就能大大缩小需要搜索的范围。

LSH 的一个重要优点是它本身有明确的理论保障。有研究提到，在参数设置合适的时候，LSH 能在亚线性时间复杂度里找出近似最近邻。

但是实际应用的时候，想要保证不错过足够多的检索结果，大多时候都要搭建很多哈希表，这样就会占用很多内存空间，另外，因为哈希函数本身就有随机性，系统还是会产生很多没用的碰撞，反而拉低了检索的速度。实验结果表明，LSH 处理高维数据的时候效果不太好。

（2）学习型哈希

为了让哈希映射的效果更好，研究者提出了学习型哈希（Learning to Hash）方法。这类方法利用机器学习技术自动学习最合适的哈希函数，能让相似的数据在哈希空间里更集中。

比较有代表性的方法有深度哈希（Deep Hashing）以及语义哈希（Semantic Hashing）。这类方法在图像检索、多媒体检索任务里已经拿到了不错的成果，不过要是用在超大规模向量检索系统里，适用范围还是比较有限的。

1.2.3 基于量化的方法

在大规模向量检索系统里，除了查询速度，存储占用也是很关键的系统指标，要是数据库规模涨到几亿甚至几十亿级别，每个向量都用原始浮点形式存的话，会给内存带来非常大的压力，举个例子，一个 768 维的向量如果用 32 位浮点数存储，那单个向量大概就要占用 3KB 的存储空间。当数据规模到 10 亿的时候，光是存这些向量本身就需要好几 TB 的内存，所以说，怎么在保住不错检索精度的同时，压缩向量表示还能降低距离计算的开销，就变成向量检索领域里一个重要的研究方向了。

量化（Quantization）方法就是为了解决这个问题被提出来的。它的主要思路是把高维连续向量映射到离散的编码空间里，这样就能大幅减少占用的存储空间，还能让相似度计算变得更快。量化方法其实就是一种向量压缩技术，利用数量不多的离散码字（Codeword）来近似代替原本的原始向量。

目前各类量化算法里，乘积量化（Product Quantization, PQ）是代表性最强、实际使用也最多的方法之一。这个算法最早由 Hervé Jégou 等人在 2011 年提出来，现在已经成为不少工业级向量检索系统（比如 FAISS）的核心技术。

乘积量化的基本思想是将高维向量分解为多个低维子空间，并在每个子空间中独立进行量化编码。设原始向量维度为 D ，PQ 首先将向量划分为 m 个互不重叠的子向量：

$$\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m] \quad (1.1)$$

其中每个子向量的维度为 D/m 。

在每一个子空间中，PQ 使用 K-Means 聚类算法学习一个码本（Codebook）。假设每个子空间包含 k 个聚类中心：

$$C_i = \{c_{i1}, c_{i2}, \dots, c_{ik}\} \quad (1.2)$$

对于数据库中的每一个向量，其第 i 个子向量 $\mathbf{x}_i$ 会被量化为距离最近的聚类中心：

$$q(\mathbf{x}_i) = \arg \min_{c_{ij}} \|\mathbf{x}_i - c_{ij}\| \quad (1.3)$$

因此，一个原始向量最终可以表示为 m 个聚类中心索引组成的编码序列：

$$\text{code}(\mathbf{x}) = (j_1, j_2, \dots, j_m) \quad (1.4)$$

其中 j_i 表示第 i 个子空间中选中的聚类中心编号。

通过这种方式，一个原本需要存储 D 个浮点数的向量，可以仅使用 m 个整数索引表示。例如：

原始向量：768 维浮点数；

PQ 参数： $m = 32$ ；

每个子空间 $k = 256$ ；

则每个子向量索引只需 8 bit 表示，总存储仅为 32bytes，相比原始向量（约 3KB），压缩率可以达到 接近 100 倍。

在查询阶段，PQ 不需要恢复完整向量，而是通过 查表方式（Lookup Table）计算近似距离。

对于查询向量 q ，首先计算其每个子向量与所有码字之间的距离：

$$d_{ij} = \|q_i - c_{ij}\| \quad (1.5)$$

这些距离可以存储在一个查找表中：

$$LUT_i[j] = \|q_i - c_{ij}\| \quad (1.6)$$

对于数据库中的某个编码向量：

$$code(x) = (j_1, j_2, \dots, j_m) \quad (1.7)$$

其近似距离可以通过简单求和得到：

$$d(q, x) = \sqrt{\sum_{i=1}^m LUT_i[j_i]^2} \quad (1.8)$$

这种方式避免了大量浮点计算，仅需查表和加法运算，因此计算速度非常快。

尽管 PQ 可以显著降低距离计算开销，但如果数据库规模非常大，仍然需要对所有向量进行扫描。因此，在实际系统中通常会结合倒排文件索引（Inverted File Index, IVF）进一步减少搜索范围。

IVF 的基本思想是先对整个向量空间进行粗粒度聚类。具体流程如下：

1. 使用 K-Means 对整个数据集进行聚类，得到 n_{list} 个聚类中心。
2. 每个数据向量只被分配到距离最近的一个聚类中心。
3. 每个聚类中心对应一个倒排列表（Inverted List）。

这样，数据库被划分为多个簇，每个簇只包含部分向量。

在查询时，系统首先找到与查询向量最近的若干个聚类中心，然后只在这些簇内进行进一步搜索，而无需扫描整个数据库。

把倒排文件跟乘积量化结合得到的 IVF-PQ 算法，是现在大规模向量检索系统里用得最多的索引结构之一，它的主要思路是在“粗筛选+精细化量化表示”的框架里做到高效检索。具体来说，IVF-PQ 先是用聚类方法（一般用 K-Means）把整个向量空间做划分，把整个数据集分成好几个不重叠的簇，每个簇都对应一个倒排列表，这样就搭好了粗粒度的索引结构，在这一步里，每个向量只会被分到离它最近的聚类中心所在的簇里，所以原始数据会被整理成多个局部紧凑的子空间，做查询的时候，系统不用把所有数据都扫一遍，只需要找到跟查询向量最接近的几个簇（数量由参数 $nprobe$ 控制），这样就能大幅缩小候选集合的大小，把搜索空间有效缩小。

做完粗筛选之后，IVF-PQ 加入了乘积量化技术，用来给簇内的向量做压缩表示。跟直接给原始向量做量化不一样，IVF-PQ 一般用“残差量化”的思路，先算出数据向量和它所属聚类中心之间的差，也就是残差向量，之后再给残差做乘积量化编码，这个方法能有效降低量化带来的误差，因为残差向量的分布一般更集中，用有限码本来做近似表示会更容易，具体来讲，残差向量会被拆成好几个低维子空间，每个子空间里都单独做聚类处理，最后每个向量都会变成一组子空间聚类中心的索引序列，利用这种编码方式，原本的高维浮点向量就能压缩成很短的离散编码，存储成本能降很多。

在做查询这一步的时候，IVF-PQ 靠查表的方法就可以很快完成距离计算，系统要先算出查询向量和各个子空间码本之间的距离表，之后扫描候选簇里的编码向量时，只要做简单的查表和累加操作就能拿到近似距离，不用恢复原始向量，也不用做成本很高的浮点计算。从候选集合里选出距离最小的 Top-K 个结果输出就行。整体来看，IVF-PQ 靠“先聚类缩小搜索范围+用量化降低计算量”这两个办法，在查询速度、存储效率还有检索精度三个方面找到了不错的平衡点，要说明的是，量化方法在实际应用里一般不会单独用，大多会和倒排结构、图索引或者其他近似搜索方法搭配使用，用来进一步提高整体的性能。而且，在处理高维大规模数据的时候，基于图的索引方法往往能在召回率和查询延迟之间得到更好的综合结果，所以它目前在工业级的向量检索系统里占主导地位。

1.2.4 基于图的方法

这些年来，利用图（Graph-based）结构做近似最近邻搜索的做法，慢慢变成了向量检索领域里的主流技术方向。跟基于空间划分、哈希或者量化的其他方法不一样，图方法不会直接给向量空间做规整的划分，反而是直接把不同数据点之间的连接关系建出来，搭出一张能用来快速搜索的近邻图结构。这类方法的主要思路是：如果不同数据点之间的相似关系，能合理地存成图里的边，那最近邻搜索的问题就变成了在图上找路径的问题，这样就能靠着图结构本身的局部连通性，快速找到要找的目标节点。

从本质上来说，图索引方法是想要搭建出一种带有“小世界性质（Small World Property）”的网络结构。在这类网络里，随便两个节点之间一般只需要少数几次跳转就能连通，而且每个节点又和自己周边的邻居保持比较紧密的连接，这种结构能让搜索算法在全局范围里快速定位方向，同时还能在局部区域做细致的搜索，这样一来就能在搜索效率和检索精度之间找到很好的平衡。

（1）基本思想与搜索机制

基于图的这类方法一般包含两个核心步骤：分别是建图（Index Construction）和图搜索（Graph Search）。

在建图的环节，算法要给数据集里的每个向量挑出若干个“邻居节点”，再连上无向或者有向的边，最后做出一张稀疏但能连通的近邻图。理想状态下，这张图得尽量贴近真实的 K 近邻图（KNN Graph），可要是精准搭建 KNN 图，需要花费的成本太高了，实际使用的算法一般会用近似的方法一步步把图做出来。

查询阶段的图搜索，一般都会用到贪心策略（Greedy Search）。简单来讲，算法得先从一个入口节点（Entry Point）开始，先计算这个节点跟查询向量之间的距离，之后会挨个访问当前节点的邻居节点；要是某个邻居节点，比当前节点跟查询向量的距离更近，算法就会移动到这个邻居节点上，再重复前面的计算和对比过程，这个过程会一直进行下去，直到找不到比当前节点更好的节点时才会停下，最后就能得到一个局部最优的结果。

这种搜索方法的核心在于，借助设计出来的图结构，让局部最优解可以逐步靠近全局最优解。所以，图本身的结构好坏会直接影响最终的检索效果。

（2）NSW^[1]：可导航小世界图

Navigable Small World (NSW) 是较早被提出来的、基于小世界网络思路的图索引方法。NSW 搭建图结构的过程一般会用增量式的方案：数据点会按顺序一个个插入进去，每插入一个新节点，都要在已经建好的图里做一次近邻搜索，找到这个节点对应的好几个最近邻，之后再建立它们之间的连接关系。

NSW 最关键的特点就是它的图结构本身就自带小世界属性，就是说它里面同时存在“短程边”跟“长程边”两种边。短程边负责连接距离近的邻居节点，对更精细的搜索有帮助；长程边则可以跨到更远的位置，方便快速跳到想要找的目标区域，正是这种结构让它的搜索复杂度差不多是按对数级增长，在实际使用的时候效率也比较高。

但是，NSW 也存在一定缺陷。比如它的图结构只有一层，缺少层级结构，处理超大规模数据集的时候，搜索路径可能会变得很长；另外，它在构建图的过程里，对数据的插入顺序比较敏感，运行稳定性不够好。

（3）HNSW^[2]：分层可导航小世界图

为了解决 NSW 的不足，研究者提出了分层可导航小世界图（Hierarchical Navigable Small World, HNSW）。HNSW 是目前最成功、应用最广泛的图索引算法之一，其核心创新在于引入了分层结构（Hierarchical Structure）。

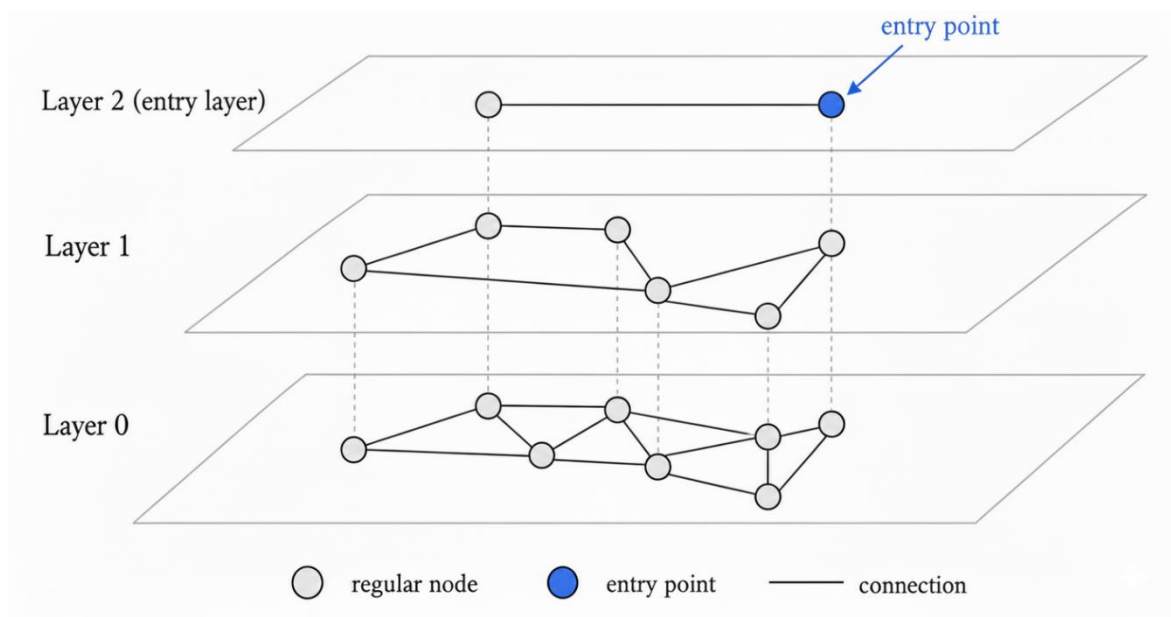


图 1.1 HNSW 索引示意图

HNSW 会把整张图分成好几个不同的层级，每个节点都会被随机分到一个最高

层级里。层级越高，节点就越稀疏，层级越低，节点反而越密集，一般来说最顶层只包含数量极少的节点，最底层则把全部数据点都包含在内了。

这种结构跟跳表（Skip List）比较相似，它主要的作用是完成“由粗到细”的搜索过程。

在高层图里，因为节点分布比较稀疏、线条也比较长，所以能快速完成全局导航，更快找到目标区域。

在低层图里，因为节点分布比较密集，可以进行更精细的搜索，进而找到精度更高的近邻结果。

查询过程里，算法会先从最高层的入口节点起步，用贪心搜索找出这一层里最好的节点，之后把这个节点当成下一层的入口，就这样一层层往下搜，一直搜到最底层，最终就能在底层得到近似最近邻的结果了。

这种“逐层收缩搜索空间”的机制，让 HNSW 在保障高召回率的同时，明显缩短了搜索路径的长度，由此能做到极低的查询延迟。

（4）NSG^[3]: Navigating Spreading-out Graph

除了 HNSW 之外，Navigating Spreading-out Graph（NSG）也是近年提出来的、比较有代表性的高性能图索引方法。NSG 的设计思路，是要在保证高召回率的前提下，造出一张更稀疏但导航能力依然不错的近邻图，以此减少内存占用，提高搜索的效率。

跟 HNSW 用的增量式多层结构不一样，NSG 一般是在提前建好的近似 KNN 图（比如用 NN-Descent 构建的）基础上再优化得到的。它的主要思路是对初始图做结构调整和剪枝处理，让最终生成的图能同时满足下面这几个关键要求。

连通性（Connectivity）：图里任意两个节点之间，都能找到路径实现互相到达；

低出度（Low Degree）：每个节点的邻居数量都有严格限制，这样就能降低存储的开销；

导航性（Navigability）：图结构需要拥有和小世界网络类似的特点，能让贪心搜索更快收敛到目标节点上。

NSG 的构建流程大概可以这样总结：第一步先选出一个全局中心点作为整个图的入口，也就是 Navigation Node，接下来从这个节点出发，对全部节点做一次广度

优先遍历（BFS），一步步把节点之间的连接关系搭建起来，在给每个节点挑选邻居的时候，NSG 不会直接保留距离最近的那几个点，而是用了围绕“相对邻域图（Relative Neighborhood Graph, RNG）思想”做出来的剪枝方法。就是，如果有一个候选邻居能通过已经确定好的邻居“间接走到”，而且走这条路径效果更好的话，那这根候选的边就会被剪掉，这样就能去掉多余没用的连接，这种剪枝策略能有效提高图的“扩展性（Spreading-out）”，让最终得到的图结构更均匀，也拥有不错的导航性能。

到了查询阶段，NSG 同样会用到基于贪心策略的图搜索方法，搜索从入口节点起步，不断访问离查询向量更近的邻居节点，一步步接近要找的目标区域。因为 NSG 图在构建的时候就已经做过细致的剪枝处理，它的搜索路径一般会更“直接”，所以在相同召回率的条件下，通常只需要访问更少的节点，也就降低了查询带来的延迟。

跟 HNSW 比起来，NSG 最主要的特点是它用了单层结构，还做到了极致的稀疏性，这样的结构让 NSG 占用的内存一般比 HNSW 更小，尤其适合那些对存储资源要求比较高的场景。但是，NSG 构建需要依赖质量很高的初始 KNN 图，整个构建过程会比 HNSW 更复杂，计算花费也更大，而且它对数据分布也更敏感，所以实际放到工业系统里用的话，HNSW 适用性更广，NSG 则只能在一些特定场景里，给出更好的性能和资源之间的平衡。

（5）DiskANN^[4]：面向外存的大规模图索引方法

随着向量检索的规模从千万级慢慢扩大到十亿甚至百亿级，只靠内存（RAM）做图索引的方法，在实际部署的时候慢慢就碰到瓶颈了，一方面，像 HNSW 这类高性能图结构，一般要存大量邻接边信息，内存花的开销挺大，另一方面，要是数据规模涨得特别多，要把整个索引全都放进内存里，基本做不到。为解决这个问题，微软给出了 DiskANN（Disk-based Approximate Nearest Neighbor）方案，这个方法的主要目标是，在借助外存（SSD）存储大规模索引的基础上，还能保住跟内存检索差不多的查询性能。

DiskANN 的主要思路，可以理解成是对图索引方法，在存储层面做了工程化的拓展，它的整体结构还是以近邻图为基础，不过在构建流程里加了不少针对外存访问特点的优化办法。DiskANN 一般会先从一张高质量的近似 KNN 图起步（思路和 NSG 差不多），再用剪枝办法做出一张出度低、连通性还很好的稀疏图，这样就能

减少每个节点需要存储的邻接信息，这一步对减少磁盘访问次数来说，是非常关键的。

第二，DiskANN 在数据排布上做了专门优化，跟传统图索引把所有节点均匀存储不一样，DiskANN 会把一部分“关键导航节点（Cache Nodes）”一直放在内存里，方便在查询开始的时候快速找到目标区域；剩下的大部分节点存在 SSD 上。当发起查询时，系统先在内存里的小型导航结构做初步搜索，之后再一步步按需要读取磁盘里的节点，这种“内存引导加磁盘扩展”的分层访问方法，能明显减少随机 I/O 的次数，进而提高整体的查询速度。

搜索策略层面，DiskANN 也使用了基于贪心的图遍历方法，为了适配外存环境，它在访问顺序和候选队列管理方面做了改进。举个例子，系统会尽量批量读取相邻的节点，降低磁盘随机读取带来的资源消耗，同时利用预取（Prefetch）机制抵消 I/O 延迟的影响，另外，DiskANN 还通过限制搜索过程中路径分支拓展的数量，让整个查询流程变得更容易控制，最终能在延迟和召回率之间找到合适的平衡点。

整体来看，DiskANN 最突出的特点就是它打破了内存容量的限制，让图索引这种方法可以扩展到超大规模的数据集里。跟 HNSW、NSG 这类主要靠内存承载数据的图方法比起来，DiskANN 只用单台机器就能支持十亿级，甚至更大规模的数据检索，同时还能维持不错的召回率，查询延迟也比较低，这样的特性让它在工业领域，尤其是搜索引擎还有推荐系统的离线大规模检索场景里，都有着很重要的使用价值。

跟别的向量检索方法比起来，以图为基础的方法有这些明显优势。

高召回率：在延迟相同的情况下，效果一般比量化方法和哈希方法更好。

查询延迟低：搜索的复杂程度接近对数级别。

动态插入的支持效果很好，比较适合需要在线更新的使用场景。

对高维数据的鲁棒性很强：不容易被“维度灾难”影响。

正因为这样，HNSW 已经成了现在主流向量数据库的核心索引结构，像 Milvus、Qdrant、Weaviate 这类系统，默认都会选用 HNSW 或是它的变体作为主要检索引擎。

当然，图方法也有它自己的局限。比方说，构建索引的成本比较高，占的内存也相对更大，一般情况下还需要额外存储邻接关系，在规模特别大的应用场景里，往往得搭配量化方法进一步把存储压缩下来，所以实际做工程系统的时候，常用的做法就是把图索引和 PQ 这类量化方法结合在一起，做成混合的索引结构，以此在精度、

速度还有存储三者之间拿到更不错的平衡。

1.3 混合检索的基本概况

伴随着向量检索技术在搜索引擎、推荐系统还有大语言模型等场景里的普遍使用，研究者慢慢发现，只靠单一的向量表示方法没办法完整描述复杂多样的用户查询想法。特别是在真实的业务场景里，用户提出的查询通常会同时包含“语义理解需求”和“精确匹配需求”这两类，这就给检索系统提出了更高要求。

稠密向量（Dense Vector）一般是深度学习模型生成的，常见的比如 BERT、BGE-M3 这类预训练语言模型，这类表示方法利用上下文建模，能抓住文本里深层的语义关系，在长文本理解、语义相似度匹配还有跨语言检索这些任务上，效果特别好。比如做语义搜索或者问答系统的时候，就算查询内容和目标文本字面差别很大，稠密向量也能靠语义空间里的相似性找到对应的结果。但是，这种“语义泛化能力”在有些场景里也会出问题。当查询要用到专有名词、产品型号、编号标识或者特定实体名称的时候，稠密向量经常没法精准区分细粒度的差别，很容易出“语义沾边但对不上”的结果。

跟稀疏向量不同，Sparse Vector 方法围绕显式词项做建模，像 BM25、SPLADE 这类模型都属于这种方法，这类方法以词表为基础，利用词频、逆文档频率等信息统计后搭建对应的向量表示，可以精准保留关键词匹配的相关信号。在需要做精确检索的场景里，比如法律文档检索、商品搜索或者日志查询这类场景，稀疏向量一般都会有更高的精度，解释性也更强，但是它的缺点是很难抓住深层的语义关系，碰到表达方式多样或者存在同义替换的查询，它的召回能力通常会比较弱。

根据上面这些观察，学界和工业界慢慢发现：稠密向量和稀疏向量在表达信息的时候，有非常明显的互补特点，稠密向量更擅长做“语义理解”，稀疏向量更擅长做“精确匹配”。为了能同时用上两者的优点，混合检索（Hybrid Retrieval）技术就出现了。

混合检索的主要思路，是把两种不同表示空间里的相似度信号融合到一起，以此搭建出更全面的相似度度量函数。最常见的实现方式里，系统会引入一个权重系数 α ($\alpha \in [0, 1]$)，给稠密向量相似度和稀疏向量相似度做线性加权。调整 α 的取值，就能在“语义相关性”和“关键词精确匹配”之间做到平滑切换，比如当 α 接近 1 时，系统会更偏向做语义检索；当 α 接近 0 时，就会更突出关键词匹配的作用，这样的

灵活性让混合检索可以适配各种各样的查询需求，在实际运行的系统里都有不小的应用价值。

如今大规模信息检索系统不断发展，混合检索不只是一种性能优化的办法，还慢慢变成了工业级搜索系统里的标准配置，在过去的搜索引擎中，倒排索引一直是主流方式，它的优点是可以高效支撑关键词匹配和布尔检索，但是，现在用户表达查询的方式越来越复杂，只靠词语匹配，已经没办法满足理解语义的要求了。尤其是在自然语言问答、长文本检索还有跨语言检索这类场景里，查询内容和文档之间通常找不到明显一样的词汇，这种时候稀疏检索方法的效果就会明显变差。

同时，用深度学习做的稠密向量检索虽然在理解语义这一块表现不错，但在实际的工业系统里也暴露出不少问题，比如在电商搜索的时候，用户输入的查询往往带着品牌名、型号或者规格参数，像是“iPhone 15 Pro 256G”这类，这种查询对精确匹配要求非常高，可稠密向量模型没办法区分开细粒度的差别，最后找出来的结果就会出问题。在法律、医疗这类专业领域里，术语必须精确匹配，只看语义相似性很可能会带来无法接受的错误。

所以，在工业领域实际落地的时候，越来越多的系统开始用“语义+关键词”的混合检索办法，比如在检索增强生成（RAG）系统里，一般得同时保证检索出来的内容既有语义相关性，又符合事实要求：稠密向量用来召回和语义相关的内容，稀疏向量则用来保证关键实体或者事实能被正确匹配上。在推荐系统里，混合检索也会被用来把用户行为的语义信息和显式标签信息结合在一起，以此提高推荐结果的精准度，同时也能让结果更多样。

从分类方法来看，现在已有的混合检索技术大概可以分成以下三类。

（1）后融合（Late Fusion）方法

也就是现在常用的“两路召回+重排”架构，这种方法把稠密检索和稀疏检索完全分开，各自独立运行，最后再把得到的结果融合到一起。它的好处是做起来简单，也能适配现有的系统，不过缺点就是会出现召回截断的问题，而且计算要花的成本比较高。

（2）前融合（Early Fusion）方法

这类方法尝试在特征层面把稠密表示和稀疏表示融合到一起，比方说把稀疏特征

放进稠密向量里，或者利用联合模型直接学习出混合表示。这类方法从理论上来说能避开多路召回的问题，但一般都要重新训练模型，工程上要花的成本比较高，而且没办法灵活地支持动态权重的调整。

（3）统一索引（Unified Index）方法

最近这些年有一类方法慢慢受到大家关注，这类方法的主要思路是做一个单一的索引结构，同时把稠密信息和稀疏信息都编码进去，这样就能在检索的时候直接完成混合相似度搜索。这类方法从理论层面看，可以避开召回截断的问题，还能降低系统的复杂程度，不过它要解决的关键问题是，怎么设计出高效、还能扩展的索引结构。

从发展趋势来说，现在的混合检索正在从“简单结果融合”慢慢往“深度结构融合”方向发展，尤其是向量数据库和大模型系统发展得越来越快，大家都觉得统一索引方法会是下一代检索系统的重要发展方向之一。怎样在保证高召回率的基础上，做到低延迟、低存储成本，还能支持动态权重调整，就是现在研究里最核心的问题。

尽管混合检索从理论层面看优势比较明显，但把它放到实际工程里落地，还是会碰到不少难题。现在工业界最常用的方案，基本都用了一套叫“两路召回+重排合并”的结构，系统会先分别在稠密向量索引（一般是图索引）还有稀疏倒排索引里各自单独做检索，两个索引分别返回 Top bk 个候选内容；之后再把这两路得到的结果合并到一起、去掉重复的内容，再按照给定的权重 α ，给候选集合里所有内容重新计算混合分数，最后排好顺序输出 Top-K 的结果。

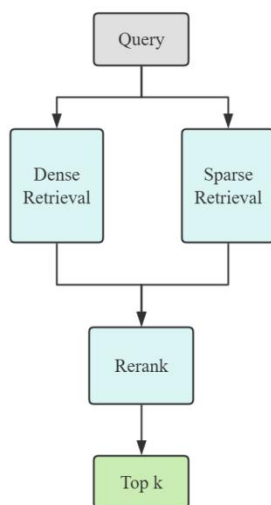


图 1.2 两路召回检索方法（baseline）示意图

但是，这套方案在实际使用的时候能看出明显的局限，主要有这么几个方面。

（1）系统工程复杂度高

这个方案要同时维护两套完全不一样的索引结构：一套用来做稠密向量检索（比如 HNSW 图索引），另一套用来做稀疏向量检索（也就是倒排索引）。这样做不光让系统实现变复杂了，在更新数据、构建索引还有管理存储这几个方面，还都会产生额外的消耗，在规模比较大的系统里，这种“两个系统分开同时维护”的做法，会大幅提高工程上的成本，也让运行维护变得更难。

（2）检索延迟显著增加

在查询阶段，系统要分别在两套索引上完成检索操作，最后整体查询的延迟差不多等于两次操作延迟加起来。在重排阶段还要对几百甚至上千个候选样本重新计算混合相似度。这里面，尤其是稀疏向量的计算，一般都要处理高维稀疏向量点积，频繁的 if-else 选择逻辑会让计算成本变高，整个流程放在高并发场景里，就会变成拖慢系统性能的瓶颈，直接影响结果的响应速度。

（3）召回率不稳定问题

这个方案最核心的问题，出在召回率不够稳定，当权重 α 取到中间数值的时候，真正符合要求的最优邻居，可能在稠密检索和稀疏检索里都挤不进前 bk 名，这样一来在第一阶段就会被截断，直接丢失掉，要是目标结果根本没进到候选集合里，就算后面的重排策略再精准，这个结果也找不回来了。虽然把 bk 调大一点能缓解这个问题，但这么做会明显增加计算需要的开销，还会让查询延迟涨得特别快，放在大规模数据的场景里根本没法用。

因此，当前的混合检索方法在检索效率、运算复杂度和召回率这三个方面依旧存在难以调和的矛盾，这也给新索引结构的设计留出了重要的研究空间。

1.4 本文所做的工作

现在已有的混合检索方法，在系统复杂程度、检索等待时间还有召回率稳定性这几个方面都存在缺陷，本文从“统一索引结构”这个方向入手，给出了一种面向混合检索的统一图索引（Unified Hybrid Graph Index）的构建和搜索方法。这个方法想把原先分开的稠密检索和稀疏检索整合到同一个图结构里面，这样就能在维持高召回率的前提下，降低系统复杂程度和查询时的等待时间，本文主要做的工作如下。

（1）混合距离函数的性质分析与理论揭示

这篇文章从理论方向入手，对混合相似度函数做了深入的分析，推导之后可以发现，采用内积形式的时候，混合距离能写成跟权重系数 α 有关的一元一次函数。也就是说 α 在连续变化的时候，不同数据点之间的相对排序会呈现分段线性的变化规律。再进一步说，实际数据里稠密向量和稀疏向量一般都存在一定相关性，由此可以推测，在一定范围里，近邻关系有比较强的拓扑稳定性。这个发现给构建统一图索引给出了重要的理论依据。

（2）统一混合图的建图方法设计

按照上面这些理论分析，本文提出了一种新的建图方法，主要思路如下：先在多个分开采样得到的 α 权重下，分别算出每个节点的 Top-K 邻居集合，再把这些邻居集合去掉重复内容之后合并起来，作为最终的邻接列表，该方法用到了“不同权重得到的邻居集合重叠性比较强”这个特点，能在邻居规模可控的前提下搭建出统一的图结构。跟传统方法比起来，这种策略可以在单个图里同时存下多种不同的相似度信息。

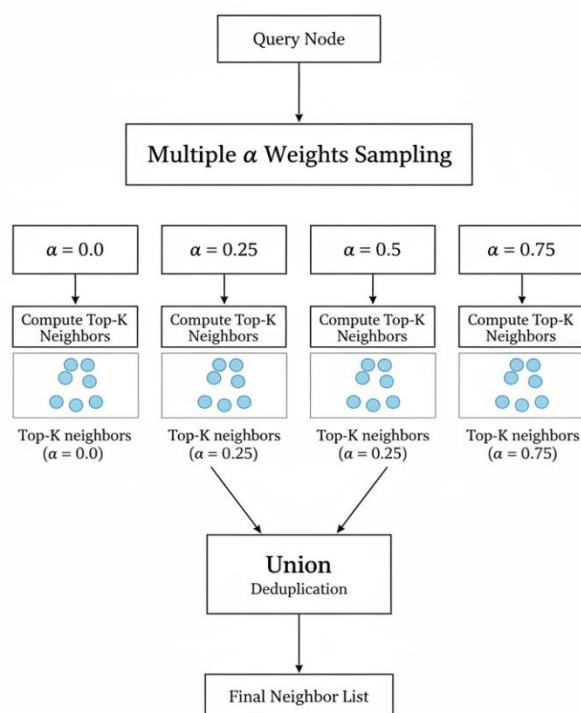


图 1.3 混合图索引（hybrid graph）示意图

（3）支持动态权重的统一索引机制

因为统一图里已经暗中包含了不同 α 权重下的邻接关系，所以查询的时候，不需要重新构建索引，就能直接支持任意的权重参数。具体来说，某一个 α 对应的搜索过程，差不多可以当成对应子图上的搜索来处理，这样就能做到“单索引、多语义”的检索能力，这个特点让系统可以灵活适配不同的查询需求，同时还能保住不错的检索效率和召回率。

（4）系统实现与工程优化

本文用 C++ 做了一个完整的向量检索系统，还针对统一图结构可能出现的问题，从多个方面做了改进。比如，为了提高图的连通性，加入了双向连边的方法；为了控制邻接的规模，减少多余的连接，用了基于相对邻域图（RNG）的剪枝办法，另外，在查询过程里还加入了基于 **lower bound** 的剪枝策略，减少了对无效节点的访问，提高搜索速度，这些工程上的调整，明显提高了整个系统的性能还有稳定性。

实验结果表明，本文提出的方法可以有效解决传统混合检索方案里存在的“召回断崖”问题。在不同 α 权重下，系统都能维持稳定且偏高的召回率，在查询延迟上也比传统两路召回方案更优，证实了本文提出的方法确实有效，也具备实用价值。实验结果表明，本文提出的方法能够有效解决传统混合检索方案中存在的“召回断崖”问题。在不同 α 权重下，系统均能保持稳定且较高的召回率，同时在查询延迟方面优于传统两路召回方案，验证了所提方法的有效性与实用价值。

2 混合检索与统一图索引方法

这一章会系统介绍针对混合检索搭建统一图索引时，涉及到的核心理论和技术基础，先从非结构化数据的向量表示开始讲，分别聊一聊稠密向量和稀疏向量的生成逻辑，以及各自的优缺点；之后给出混合检索在内积空间里严格的数学定义，再深入拆解传统“两路召回+重排”Baseline 架构，在运行耗时和召回率上存在的根本问题。结合前面提到的这些理论层面的问题，详细推导并给出本文提出的统一混合近邻图（Unified Hybrid Graph）的算法设计。

2.1 数据的向量表征

在现代信息检索与自然语言处理系统中，如何将人类可读的非结构化文本转化为计算机可高效计算的数学表示，是检索系统的第一步。文本的向量表征模型直接决定了高维向量空间中语义距离度量的物理意义。目前，文本表征技术主要分为两大阵营：稠密向量（Dense Vector）与稀疏向量（Sparse Vector）。

（1）稠密向量表征（Dense Vector）

稠密向量的作用，就是把长度不定的文本，映射到一个维度固定的连续低维实数空间 $\mathbb{R}^d$ 里。它的维度 d 一般在 128 到 4096 这个范围里面，在这个空间里，向量每个分量都带着非零的实数值，这些分量不会和文本里的某个特定字或者词对应上，而是深度神经网络（比如基于 Transformer 的各类架构）对整段文本序列做全局编码后，提取出来的潜在语义特征。

常见的稠密表征模型有 BERT、Sentence-BERT、E5，还有本文实验用的 BGE-M3 模型，稠密向量最核心的优点，就是它有很强的“语义泛化能力”，哪怕查询语句和目标文档在字面用词上一点都不一样，只要两者表达的深层意思差不多（比如换成了同义词、换了句子说法这类情况），它们在稠密向量空间里的夹角就会很小。但这种很强的泛化能力也带来了“语义漂移”的问题，碰到需要查询特定专有名词、产品序列号或者精准数字匹配这类任务时，稠密模型经常会漏掉这些局部的关键信息，导致它做精确匹配的能力不够好。

（2）稀疏向量表征（Sparse Vector）

稀疏向量是传统统计信息检索，比如 TF-IDF、BM25 这类方法的自然延伸，近些年还得到了深度遮蔽语言模型（Masked Language Model, MLM）的明显助力，比如 SPLADE 模型就是这类代表。稀疏向量的维度 d_{sparse} 和整个词表的大小是一样的，通常能达到几万甚至几十万维，但是，随便拿一段具体文本来看，绝大多数词汇都不会在里面出现，所以这类向量里超过 99% 的分量都是零。

对于文本 T ，其稀疏表示可以定义为 $V_{\text{sparse}} \in \mathbb{R}^{d_{\text{sparse}}}$ 。其中非零分量的权重可以由传统的词频统计决定，也可以由 SPLADE 等深度模型输出的对数几率决定。稀疏向量的核心优势在于极强的“精确匹配能力”和“词汇级可解释性”。只有当查询和文档在具体的词汇（或模型激活的高频同义词）上发生直接重叠时，才会产生非零的内积得分。这使得稀疏向量在处理关键词检索时具有不可替代的地位。

以文本表示为例，假设构建一个词表 $V = \{\text{"I"}, \text{"love"}, \text{"machine"}, \text{"learning"}, \text{"cat"}\}$ ，对于句子“I love machine learning”，基于词袋模型可以得到一个 5 维向量，如按词频表示为[1, 1, 1, 1, 0]；若词表扩展到上万维，而句子中仅包含少量词，则向量形式类似于[0, 0, ..., 3, ..., 1, ..., 0]，即绝大多数维度为 0，仅少数位置为非零值，这种“高维且大部分为零”的表示即为稀疏向量。需要指出的是，稀疏向量的非零值在实际中并不局限于 1，而可以是词频或加权结果，例如采用 TF-IDF 时，向量可能表示为[0.1, 0.2, 0.8, 0.9, 0, ..., 0]，其中数值大小反映词语在当前文本中的重要程度。相比之下，稠密向量通常由神经网络训练得到，例如通过 Word2Vec 将词语表示为低维实数向量，如“machine”可表示为[0.21, -0.13, 0.78, 0.05, -0.44]，其各维度几乎均为非零值；类似地，“cat”和“dog”的向量在数值空间中往往较为接近，从而体现语义上的相似性。总体而言，稀疏向量具有维度高、非零元素少、结构直观的特点，而稠密向量则具有维度低、信息分布集中、语义表达能力更强的优势。

综上所述，稠密向量与稀疏向量在捕捉文本语义与匹配精度上展现出了完美的互补性，这构成了混合检索技术不可或缺的底层数据基础。

2.2 混合距离度量

由于稀疏向量距离计算采用 IP 内积距离，本文统一采用内积（Inner Product, IP）作为底层的相似度度量基准。引入混合权重 α ，混合 IP 内积表示为：

$$IP_{hybrid} = \alpha * IP_{dense} + (1 - \alpha) * IP_{sparse} \quad (2.1)$$

在向量检索中，距离越小表示向量间的相似度越大，所以混合距离定义为：

$$D_{hybrid} = 1 - IP_{hybrid} \quad (2.2)$$

该距离函数是贯穿本文统一图构建与图上搜索的核心数学指标。

2.3 Baseline 架构分析

2.3.1 Baseline 的耗时分析

Baseline 方案包含稠密独立召回、稀疏独立召回与精确重排三个阶段。设稠密路检索前 bk 个候选的耗时为 T_{dense} ，稀疏路检索前 bk 个候选的耗时为 T_{sparse} 。合并候选集后，对其进行去重和基于动态 α 的全量精确距离重排，耗时记为 T_{rerank} 。该方案的系统总查询延迟 T_{total} 可表示为：

$$T_{total} = T_{dense} + T_{sparse} + T_{rerank} \quad (2.3)$$

可见，当需要保证召回率时，两路检索的代价会明显上升，尤其是 T_{sparse} ，导致总耗时上升。

2.3.2 Baseline 的召回分析

除了延迟高昂，Baseline 最致命的缺陷在于中间 α 值下的召回率骤降。假设系统最终需要返回 Top k 个结果。对于查询 q ，假设存在一个真实目标点 x^* 。当用户指定 $\alpha=0.5$ 时， x^* 确实拥有前 Top k 的混合距离，应当被成功召回。然而，在 Baseline 的第一阶段召回中，由于稠密和稀疏两路是独立工作的，它们只能分别依据稠密距离和稀疏距离进行排序。如果 x^* 是一个“各项均衡但单项不拔尖”的数据点，它在稠密和稀疏两路都未被检索到（尽管它入选了最终的混合 Top k ）此时，单路召回的硬性截断导致，目标点在合并候选池中被提前过滤。无论后续的重排算力多么强大，丢失的节点永远无法找回。为了填补这一召回率上的损失，工程上只能被迫将 bk 放大到数千甚至上万，但这又将导致两路检索开销显著上升，无法同时兼顾召回率与检索效率。

2.4 混合图算法设计

针对传统 Baseline 架构多路割裂带来的严重缺陷，本文提出了一种面向混合检索的统一图索引（Unified Hybrid Graph）构建与搜索算法。该算法将全 α 范围的近邻拓扑结构压缩进单一的图中，在查询阶段实现无重排的高效图上游走，在任意 α 权重下搜索都能够近似于在对应的子图上搜索。

2.4.1 混合图算法设计的数学直觉

在混合 IP 内积计算公式（2.1）中，观察到混合 IP 内积是关于 α 的一次函数，故随着 α 从 0 平滑滑动到 1，空间内任意两点相较于目标点的“排序胜负”只会在某个特定的临界点发生一次翻转，即对于图上的某点而言，它的 Top k 邻居是随 α 线性变化的。从边的视角看，考虑每个 α 下的 knn 图，每条边的“生存区间”（表示当 α 落在该区间时此边存在）也是线性变化的。从图形的视角来看，以 α 为横轴，混合 IP 内积为纵轴，对于某一点 x_i 而言，到其它 $n-1$ 个点的 IP 内积对应了 $n-1$ 条直线，这 $n-1$ 条直线在某个 α 下处于最顶部的 Top k 条构成了该 α 下点 x_i 的 Top k 邻居，随着 α 的变化，这些 Top k 集合也在变化，但这变化是线性的，Top k 邻居只会在间断点（直线的交点）处发生变化。而考虑到：

- 1) : k 是有限的
- 2) : 讨论的区间是有限的 ($\alpha \in [0, 1]$)
- 3) : 稠密和稀疏具有一定相似度（正相关性），这些直线的斜率是近似的

故猜测引起 Top k 变化的间断点应是有限且可接受的，从而如果构建统一的混合图索引，即把所有 α 下的 knn 图合并得到的混合意义下的统一的近邻图，其邻居数也是可接受的，也就是说建图是实际可行的。初步实验中，统计结果表明对上述 knn 图进行合并后，得到的混合近邻图的平均度数约小于 $2k$ ，故而从实验上也证明了混合图构造算法的可行性。而如此构建得到的混合图，是所有 α 下的 knn 子图的合并，包含了任意 α 下的邻居信息。对于任意 α ，在该混合图上检索都近似于在该 α 下的 knn 图上搜索，从而用一张图就能够实现对动态 α 权重的针对性检索。

图 2.1 中举例展示了某点到其他 $n-1$ 个点的混合距离，对应 $n-1$ 条关于 α 的一次函数，随着 α 在 $[0, 1]$ 区间上变化的示意图。假设共有 5 条直线，对应其余 5 个点，需要从中选 top3 作为最终邻居，不同 α 下的 top3 邻居是不同的，随着 α 变化而线

性变化，变化的点即为图中的 breakpoint。

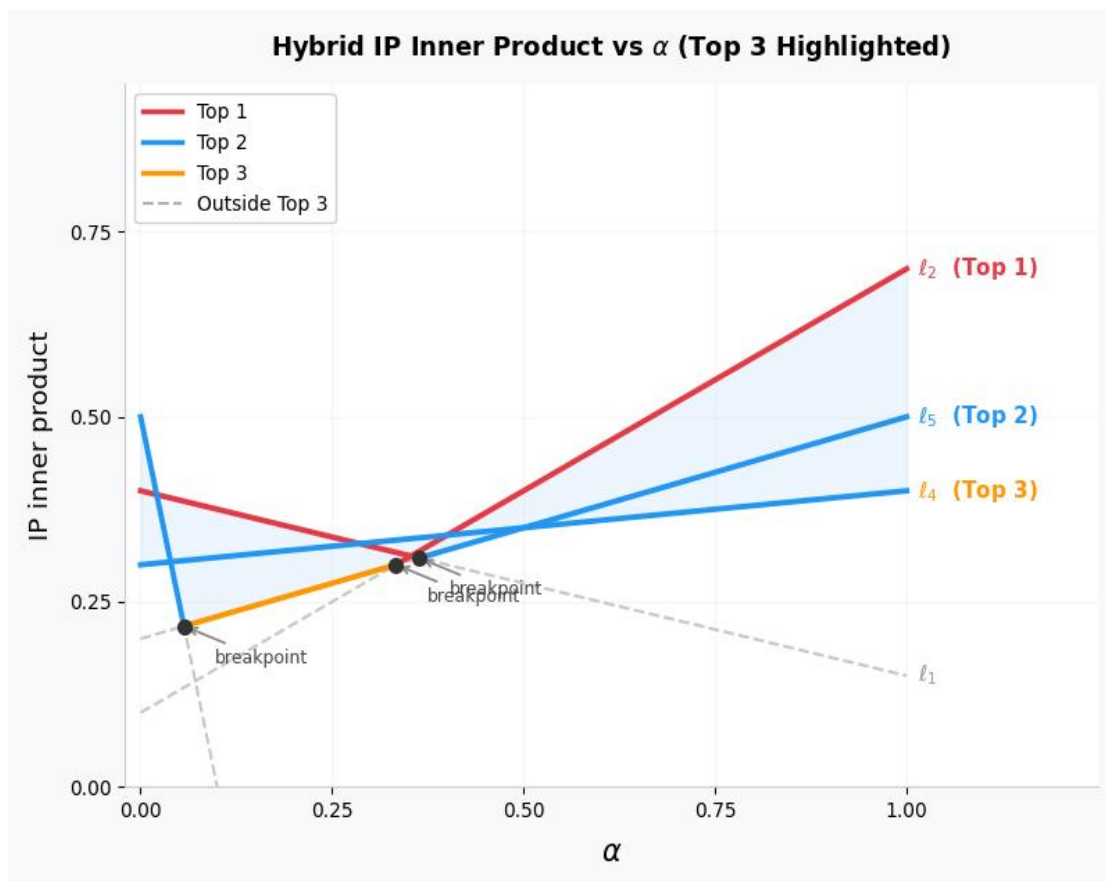


图 2.1 $n-1$ 条直线在不同 α 下的 top k 变化示意图

2.4.2 统一混合近邻图的建图算法

虽然理论上通过求并集可以构建理想的统一近邻图，但如果在百万级甚至更大规模数据上对所有的 α 采样值逐一执行全量暴力搜索（Brute-force Search），其建图的时间复杂度将呈现 $O(|\text{Alpha_set}| \cdot N^2)$ 的爆炸式增长，这在工业界的算力和成本约束下是不可接受的。

为此，本文设计了一套基于双路候选池加速的近似建图策略（见图 1.3），在计算每个采样 α 下的 top k 邻居时，不全量计算所有数据点，而是先用稠密索引和稀疏索引分别进行一次检索（两路检索的结果数应远大于 k ），取两路召回的结果作为候选池代替全局候选，将全局的暴力距离计算转化为局部的精确重排，从而极大减少了构图时间，且在精度和效果上几乎近似于全量精确计算。同时在构图最后，还加入了可选的连反向边、RNG 剪枝、DFS 兜底连通等优化策略，以提高图的连通性并减少

图半径等，其具体流程如下：

1) 构建单路基准索引：分别构建数据集的稠密索引 I_{dense} 和稀疏索引 I_{sparse} 。

2) 候选池扩张与生成：针对数据集中的每一个数据点 x_i ，将其作为查询点，分别在 I_{dense} 和 I_{sparse} 中检索出 Top ak 个最近邻点（其中 a 为候选池扩展系数， $a \gg 1$ ，如 $a = 10$ ）。将这两路返回的结果合并并去重，生成该点的局部候选池：

$$C_i = C_{dense} \cup C_{sparse} \quad (2.4)$$

实验证明，该候选池包含了极大部分在任何中间 α 状态下的真实 Top k 近邻，而其规模被严格限制在约 $2ak$ 以内，以极小的精度损失换取了极大的速率提升。

3) 动态权重采样与精确重排：定义离散采样集合

$$S_{\alpha} = \{0, \delta, 2\delta, \dots, 1\} \quad (2.5)$$

针对每一个采样点 α ，在缩小的候选池 C_i 内部，精确计算 x_i 与池中所有点的高维混合距离并排序，截取局部 Top k 的邻居，记为 Top_k_alpha 。

4) 邻居合并：数据点 x_i 在最终统一近邻图 G 中的有向边（即邻居表），定义为所有 α 采样下的邻居的并集。

5) 优化算法：由上述步骤得到初步统一混合图后，首先连反向边提高图的连通性，防止部分点因没有入度而导致无法搜索到的情况；然后再用 RNG 剪枝，删除不重要的边并引入长边，从而提高检索效率。最后用 DFS 做一个兜底连通，防止存在部分孤立点的问题。这部分优化算法是可选的，会提高建图时间，带来一定的效果提升。

算法举例说明如下：假设当前数据集中包含 5 个节点：A、B、C、D、E，每个节点同时具有稠密向量表示和稀疏向量。对于某一节点 A，在不同混合权重 α 下，其近邻关系会发生变化。例如，当 $\alpha = 0$ （完全依赖稀疏相似度）时，A 的 Top-2 邻居可能为 {B, C}；当 $\alpha = 1$ （完全依赖稠密相似度）时，其 Top-2 邻居变为 {D, E}；而在 $\alpha = 0.5$ 时，其 Top-2 邻居可能为 {B, D}。按照本文提出的建图方法，对多个 α 进行采样，计算候选池中所有点的混合距离，最终选取当前 α 下的 k 近邻，将各个 α 下得到的邻居集合取并集，最终 A 的邻居为 {B, C, D, E}。对所有节点重复该过程，即可构建出统一混合近邻图，该图中隐式包含了不同 α 条件下的多种近邻结构。

建图算法的伪代码如下：

Algorithm 1 Unified Hybrid Graph Construction

Require: Dataset $X = \{x_1, x_2, \dots, x_n\}$

- 1: Dense index I_{dense} , Sparse index I_{sparse}
- 2: k (number of neighbors), a (expansion factor)
- 3: $_set = \{0, \delta, 2\delta, \dots, 1\}$

Ensure: Graph G

- 4: Initialize $G \leftarrow \emptyset$
- 5: **for** each $x_i \in X$ **do**
- 6: $C_{dense} \leftarrow \text{Search}(I_{dense}, x_i, \text{top } a \cdot k)$
- 7: $C_{sparse} \leftarrow \text{Search}(I_{sparse}, x_i, \text{top } a \cdot k)$
- 8: $C_i \leftarrow C_{dense} \cup C_{sparse}$
- 9: $N_i \leftarrow \emptyset$
- 10: **for** each $\alpha \in _set$ **do**
- 11: **for** each $x_j \in C_i$ **do**
- 12: $d \leftarrow \text{HybridDistance}(x_i, x_j, \alpha)$
- 13: **end for**
- 14: $Top_k^\alpha \leftarrow \text{top } k \text{ elements in } C_i \text{ by } d$
- 15: $N_i \leftarrow N_i \cup Top_k^\alpha$
- 16: **end for**
- 17: Add directed edges $(x_i \rightarrow x_j)$ for all $x_j \in N_i$ into G
- 18: **end for**
- 19: **// Add reverse edges**
- 20: **for** each edge $(x_i \rightarrow x_j) \in G$ **do**
- 21: **if** $(x_j \rightarrow x_i) \notin G$ **then**
- 22: Add edge $(x_j \rightarrow x_i)$ into G
- 23: **end if**
- 24: **end for**
- 25: **// RNG pruning**
- 26: **for** each $x_i \in G$ **do**
- 27: $N_i \leftarrow \text{RNG_Prune}(N_i)$
- 28: **end for**
- 29: **return** G

图 2.2 统一混合近邻图的建图算法

2.4.3 统一混合近邻图的搜索算法

（1）自适应动态 Alpha 贪心搜索策略

当用户发起查询时，给定查询向量 q 以及用户实时期望的混合权重 α 。系统在图 G 上执行贪心搜索策略。

贪心搜索策略和其他图索引的搜索策略一样，算法在内存中维护两个关键的优先队列：

候选集最小堆（Candidates Min-Heap）：按距离从小到大排序，堆顶始终是当前发现的、距离 q 最近且尚未被展开评估的候选节点，用于指导下一步图游走的拓展

方向。

结果集最大堆 (Results Max-Heap): 容量被严格限制为搜索束 (Search Beam Size, ef)，按距离从大到小排序。堆顶元素代表了当前我们找到的最优集合中“最差（最远）”的节点距离。它构成了搜索剪枝和提前终止的动态阈值。当候选堆中的最近元素小于结果堆中的 Top ef 时，搜索终止，从结果堆中返回 Top k 。

无论用户传入的动态 α 是多少，算法不需要做任何索引结构上的改变，无需重构索引，采用完全相同的搜索流程却能够达到在动态 α 的子图上进行搜索的效果，故称为自适应的贪心搜索策略。由于统一图的邻接表中已经“预埋”了该 α 对应的最优导航边，贪心搜索能够天然地沿着这些高优边快速逼近目标空间。

(2) 搜索剪枝

由于距离计算会带来严重的时间开销，尤其是稀疏距离的计算，本算法在搜索时也加入了剪枝策略。当计算查询点 q 与某点 x_i 的混合距离时，先算稠密距离，再算稀疏距离，当结果堆已满时，如果稠密距离的结果已经大于结果堆中的 lower bound，则可以认为该点已经不太符合要求，不再计算该点的稀疏距离，即用稠密距离剪枝掉对稀疏距离的无用计算。反过来当需要剪枝稠密距离计算时（如数据集的稀疏距离远大于稠密距离时，用稀疏可以剪枝掉更多的无用点），也可以先算稀疏距离，再算稠密距离，用稀疏距离剪枝稠密距离。实验结果表明，剪枝策略在对召回率影响较小的同时可以明显提升搜索速率。

算法举例说明如下：假设查询向量为 q ，现要检索出其最近邻节点 A ，且用户指定 $\alpha = 0.5$ 。算法首先从入口节点（例如 E ）开始，计算其与 q 的混合距离，并将入口点加入到候选节点列表与结果堆中；随后访问候选节点列表中与 q 最近的节点，即节点 E ，扩展其邻居节点（如 A 、 D ），计算它们与 q 的混合距离，并加入到候选节点列表与结果堆中，再同样取候选节点列表中未扩展过邻居且离 q 最近的节点，假设为节点 A ；接着继续访问 A 的邻居集合 $\{B, C, D, E\}$ ，计算它们与 q 的混合距离，然后加入到候选节点列表和结果堆中（结果堆大小为 ef ，若堆已满需要和堆中的 lower_bound 比较再决定是否加入结果堆）……当候选堆中的最优节点已经无法优于当前结果堆中的最差节点（lower_bound）时，搜索过程终止，最终返回 Top- k 个近邻结果。整个过程可以看作是在统一图上进行一次“带方向的贪心游走”，而不同 α 只

会影响距离计算方式，并不会改变图结构本身。

搜索算法的伪代码如下：

Algorithm 2 Adaptive Hybrid Graph Search

Require: Query q , Graph G , k , ef , α
Ensure: Top- k nearest neighbors

- 1: Initialize:
- 2: $Candidates \leftarrow$ min-heap
- 3: $Results \leftarrow$ max-heap with capacity ef
- 4: $Visited \leftarrow \emptyset$
- 5: $entry \leftarrow$ random node
- 6: $d_{entry} \leftarrow \text{HybridDistance}(q, entry, \alpha)$
- 7: push $(entry, d_{entry})$ into $Candidates$
- 8: push $(entry, d_{entry})$ into $Results$
- 9: add $entry$ into $Visited$
- 10: **while** $Candidates$ is not empty **do**
- 11: $current \leftarrow$ pop from $Candidates$
- 12: $worst \leftarrow$ max distance in $Results$
- 13: **if** $current.distance > worst$ **then**
- 14: **break**
- 15: **end if**
- 16: **for** each neighbor nb of $current$ in G **do**
- 17: **if** $nb \in Visited$ **then**
- 18: continue
- 19: **end if**
- 20: add nb into $Visited$
- 21: $d_{dense} \leftarrow \text{DenseDistance}(q, nb)$
- 22: **if** $Results$ is full and $d_{dense} > worst$ **then**
- 23: continue
- 24: **end if**
- 25: $d_{sparse} \leftarrow \text{SparseDistance}(q, nb)$
- 26: $d \leftarrow \text{Combine}(d_{dense}, d_{sparse}, \alpha)$
- 27: push (nb, d) into $Candidates$
- 28: **if** $Results$ not full **then**
- 29: push (nb, d) into $Results$
- 30: **else if** $d < worst$ **then**
- 31: pop worst from $Results$
- 32: push (nb, d) into $Results$
- 33: **end if**
- 34: **end for**
- 35: **end while**
- 36: **return** top- k elements from $Results$

图 2.3 统一混合近邻图的搜索算法

3 实验设计与评估方法

为了全面验证本文提出的统一混合近邻图（Unified Hybrid Graph）在混合检索任务中的有效性与优越性，本文基于开源高性能向量检索库 `vsag` 构建了完整的实验系统。实验从检索精度与系统效率两个维度出发，对比分析了传统“两路召回+重排”Baseline 方法与本文方法的性能表现，并通过多组参数实验系统评估不同配置下的性能变化趋势，从而验证所提方法在实际应用中的稳定性与可扩展性。

3.1 实验环境与数据集

3.1.1 实验环境

实验在如下软硬件环境中运行：

操作系统：Linux x86_64

CPU：Intel Xeon Platinum

内存：128GB

编译器：GCC 9.2.1

构建工具：CMake 3.26.4

代码仓库：`vsag`^[5]

在实现层面，本文基于 `vsag` 向量检索框架进行开发，并在其基础上实现统一混合图索引结构与对应的搜索算法。所有实验均在相同环境下运行，避免硬件差异对结果造成影响。

3.1.2 数据集处理

为保证实验结果的代表性，本文选用信息检索领域广泛使用的 `MSMARCO`^[6] 数据集作为评测基准。该数据集来源于真实搜索日志，具有较强的实际应用背景，能够较好反映混合检索场景下的性能表现。

实验中使用的数据规模如下：

- 文档数量：约 80,000 条（Docs）
- 查询数量：1,000 条（Queries）

在做向量化处理的时候，统一用预训练模型 `bge-m3` 给文本做编码，同时生成稠

密向量跟稀疏向量两种表达。这里面，稠密向量用来抓取文本里的语义信息，稀疏向量用来保留关键词匹配的特征，这样就能搭出一个完整的混合检索表示空间。

为了检测检索算法的准确程度，本文用到全量暴力计算，也就是 Brute-force Search 的方法，给每个查询生成真实的 Top-k 最近邻结果，也就是 Ground Truth，具体来看，对每个查询向量，都分别算出它跟所有数据点之间的混合距离，再做排序得到精准结果。把数据和对应的 Ground Truth 统一存成 HDF5 格式，方便快速加载数据、重复做实验。

3.2 评价指标与对比基线

3.2.1 评价指标

为了从精度与效率两个方面全面评估检索性能，本文选取如下评价指标：

（1）召回率（Recall@k）

表示系统返回的 Top-k 结果中包含真实 Top-k 最近邻的比例，是衡量检索准确性的核心指标。Recall 值越高，说明系统能够更好地找到真实邻居。

（2）查询吞吐量（QPS, Queries Per Second）

这个指标表示系统单位时间里能处理的查询数量，用来衡量检索的快慢效率。放到实际系统里，QPS 会直接影响系统的响应能力以及同时处理多个请求的能力。

做实验的时候，调整不同算法里的关键参数（比如 Baseline 里的 bk 和本文方法里的 ef），就能得到不同精度和效率搭配下的性能点，之后画出 Recall-QPS 曲线，就能直观看出来不同方法在“精度-效率”平衡上的区别。

3.2.2 baseline 方案设定

分别围绕稠密向量和稀疏向量搭建各自独立的索引结构，实验里两路索引都用了 vsag 中的 HGraph，也就是基于 HNSW 的改进版来搭建，以此保证单路检索的性能处在较好水平。

进入查询阶段后，系统会分别在两个索引里各自独立做检索，每一路都会返回 Top-bk（为保证两路检索的召回率，bk 一般设置为大于 k 的值）个候选结果。之后把两路的候选结果合并到一起去掉重复项，再在统一的候选池里按照预先设置的混合权重 α 算出精准的混合距离，最后做完全部重排工作，返回 Top-k 结果。

这个方法可以较好地把稠密信息和稀疏信息结合起来，但因为它要依靠多阶段的处理流程，在计算的资源消耗和召回稳定性上都有一些缺点，所以本文把它当成主要的对比对象。

装
订
线

4 实验结果与性能分析

4.1 实验结果

实验在固定返回结果规模 $k=200$ 的条件下，在多个混合权重 α （0.2 - 0.8 之间采样， α 趋于 0 或 1 时退化成单一稀疏检索或稠密检索，不是本文所讨论的范围）下对所提出方法与 Baseline 方法进行了系统对比实验。实验中计算 recall 的 ground truth 是通过提前暴力搜索对应 α 下 query 到 doc 的混合距离得到。计算 qps 时，baseline 的两路召回是串行而非并行计算。通过调节参数 bk（Baseline）与 ef（Hybrid Graph），获得不同精度-效率组合下的性能表现，并绘制对应的 Recall-QPS 曲线。完整实验数据如下所示：

表 4.1 baseline 与 hybrid graph 的 recall-qps 数据

			bk=ef=200	bk=ef=300	bk=ef=400	bk=ef=500
$\alpha=0.2$	baseline	recall	0.84187	0.933613	0.964863	0.977713
		qps	177.944	127.845	98.8191	80.6614
	hybrid graph	recall	0.921949	0.949893	0.963377	0.971298
		qps	195.366	140.149	111.819	92.8814
$\alpha=0.4$	baseline	recall	0.839255	0.932789	0.967123	0.981318
		qps	176.36	124.064	97.7014	78.5914
	hybrid graph	recall	0.937329	0.962818	0.974388	0.980917
		qps	209.774	151.879	123.578	102.013
$\alpha=0.6$	baseline	recall	0.88236	0.959718	0.979343	0.986668
		qps	178.624	127.095	95.0245	80.313
	hybrid graph	recall	0.939143	0.964483	0.976283	0.982992
		qps	218.328	158.749	127.558	107.021
$\alpha=0.8$	baseline	recall	0.920269	0.966692	0.978467	0.984862
		qps	179.288	123.132	95.4466	78.695
	hybrid graph	recall	0.934158	0.961792	0.974553	0.982142
		qps	223.173	162.381	128.124	108.31

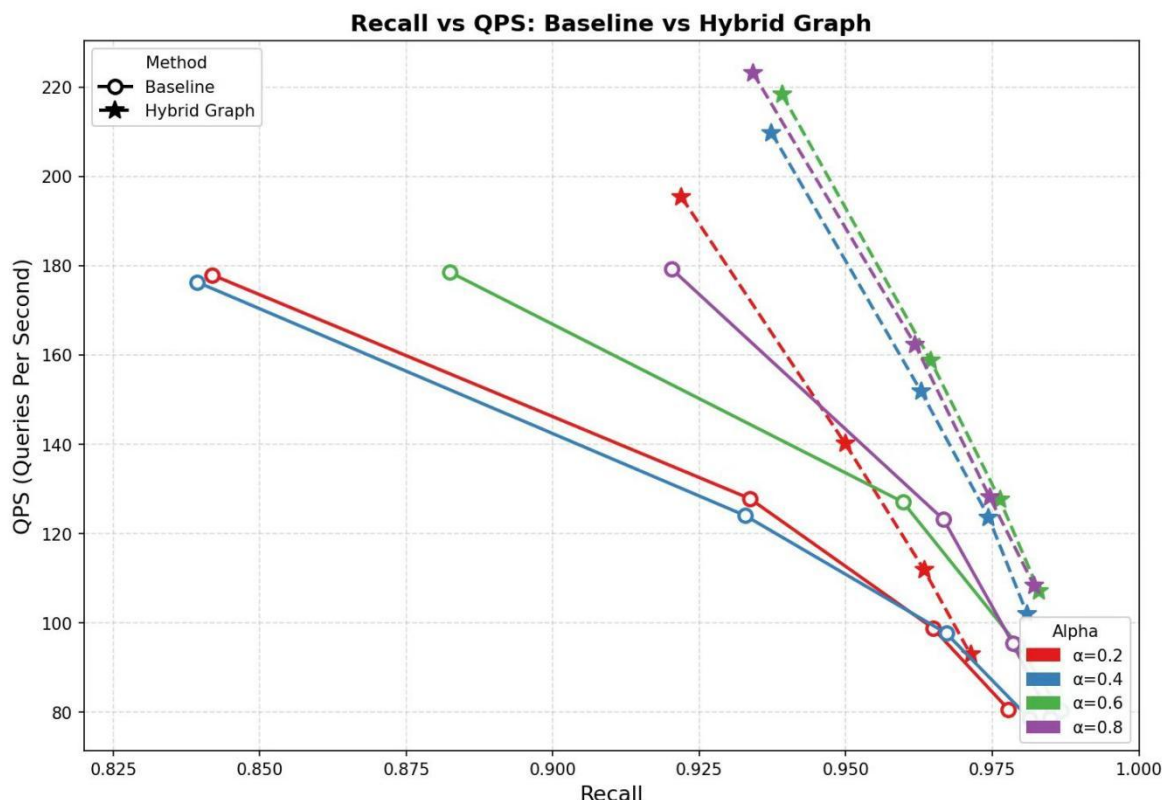


图 4.1 baseline 与 hybrid graph 的 recall-qps 曲线对比

4.2 性能分析与讨论

（1）从整体性能上来看，混合图索引方法优于 baseline 方法。实验结果表明，固定返回规模 $k=200$ 的时候，本文在多个混合权重 α （0.2–0.8）下，把提出的统一混合图索引和传统“两路召回+重排”Baseline 方法做了全面对比。结果表明，本文方法不管是召回率还是检索效率，这两个主要指标上都有明显优势，而且在不同权重设置下性能表现也更稳定。

（2）在计算预算比较低的时候，本文所提的混合图索引方法优势要更突出。当 $bk=ef=200$ 时，所有 α 取值里，统一混合图的召回率都比 Baseline 高不少。比如 $\alpha=0.2$ 和 $\alpha=0.3$ 这种更偏向混合语义的情况，Baseline 的召回率大概只有 0.84 和 0.83，本文方法可以提到大约 0.92 和 0.93，提高的幅度有 8%–10%，这个结果也能说明，若 bk 不是很大的情况下，Baseline 在第一阶段分开召回的过程里，明显存在信息丢失的问题：有些在混合距离计算里本该进到 Top- k 的“均衡型邻居”，没办法同时进入稠密和稀疏两路的候选集合，到后面阶段也找不回来；但本文方法靠统一的图结构提前把不同 α 下的邻接关系融合在一起，刚好解决了这种“召回截断”的问题，所以就算参数

设置比较低，也能保住比较高的召回率。

(3)混合图索引的检索效率更高。随着参数规模变大(bk 或 ef 从 200 提到 500), **Baseline** 的召回率慢慢接近本文方法,但它还是要付出更高的计算成本,这一点在 **QPS** 指标上看得特别清楚:相同召回率水平下,统一混合图一直都有更高的查询吞吐量,比如 $recall$ 接近 0.96 的范围里,本文方法的 **QPS** 一般比 **Baseline** 高出 10%–25%左右。追根究底是因为 **Baseline** 要走“两路分开检索 + 候选合并 + 全量重排”的多步骤流程,稀疏向量计算和重排这两步都多花了不少算力;但本文方法把多步流程压成了一次图搜索,既避免重复计算,也明显降低了查询的延迟。

(4)混合图索引面对不同的混合权重能保持稳定的高性能。对不同 α 区间做进一步分析能发现,本文方法的优势在中间权重 ($\alpha \approx 0.3-0.6$) 范围内最明显,这个结果和之前的理论分析对上了:在这个区间里,稠密信号和稀疏信号都发挥很重要的作用,真实的近邻往往不是在单一空间里最优,而是“综合起来最优”,**Baseline** 因为靠单路排序做截断,最容易在这个区域出问题,该召回的没召回;而统一混合图在建图的时候就已经融合了不同权重下的邻接关系,让这些“跨空间最优”的节点能保留在图结构里,查询的时候就能顺利访问到。也因此,本文方法在不同 α 下都能保持更稳定的召回率,**Baseline** 则会出现明显的性能波动。

(5)混合图索引在较为极端的混合权重下仍优于 **baseline**。另外,当 α 接近 1,也就是更偏向稠密检索的时候,两种方法的召回率差距会变小,这是因为 **Baseline** 方法本身的稠密索引就已经有不错的性能了。但是哪怕是在这种条件下,本文提出的方法在查询效率方面依旧有着稳定的优势,这就能说明统一图结构不光在混合检索场景里能用,哪怕在接近单一路径检索的场景中,也能有不错的性能表现。

整体来看,本文方法跟 **Baseline** 方法相比,改进主要在三个地方:第一,利用统一图结构去掉了“两路召回”带来的候选截断问题,从根本上提高了召回率;第二,用单阶段图搜索代替原来的多阶段检索加重排流程,让查询效率变高,第三,在建图阶段就给多权重邻接关系做好编码,让系统可以很自然地支持动态 α 查询,不管是什么样的检索需求,都能保持稳定性能。实验结果充分证明了混合图索引方法的有效性,跟传统的 **Baseline** 方法比,混合图索引有着明显的整体优势。

5 结论和展望

5.1 研究结论

本文围绕混合检索场景里“稠密向量和稀疏向量融合”这个主要问题，针对传统“两路召回+重排”架构在系统复杂度、检索效率和召回稳定性上存在的不足，给出了一种面向混合检索的统一图索引构建方法，还做了系统性的理论分析和实验验证。主要结论如下。

(1) 针对混合相似度函数的特性做了理论分析，发现在内积形式下混合距离对权重参数 α 来说是一元一次函数关系，不同数据点的排序变化带有分段线性的特点。这个性质能说明，不同权重下的近邻关系有一定的拓扑稳定性，给统一索引结构的构建给出了重要的理论依据。

(2) 提出了一种统一混合近邻图 (Unified Hybrid Graph) 的建图方法。这个方法利用在多个离散权重下采样，再合并各自的 Top-K 邻居集合，把不同语义空间下的近邻关系统一放到单个图结构里编码，最终达成“单索引、多语义”的表示效果。

(3) 设计并实现了支持动态权重查询的统一检索机制

不用重建索引，也不用执行全量重排，系统就能直接在统一图上完成搜索，做到对任意 α 权重的高效支持，大幅降低查询阶段的计算复杂度。

(4) 在工程实现层面，本文围绕 C++ 搭建了完整的向量检索系统，利用引入双向连边、基于相对邻域图 (RNG) 的剪枝策略以及 lower bound 剪枝优化的方式，有效提高了图结构的连通性和搜索效率。

(5) 在 MSMARCO 数据集上做了实验评估之后能看到，本文方法在多个 α 权重下的表现都比传统 baseline 更好。在召回率 (Recall) 和查询吞吐量 (QPS) 这两个关键指标上，统一混合图都拿到了更优的结果，也验证了该方法在效率和效果上的综合优势。

因此来看，本文提出的统一图索引方法，有效解决了传统混合检索方案里系统复杂度、查询延迟大以及召回率不稳定这类问题，不管是理论分析还是工程实践层面，都有不错的研究价值和应用潜力。

5.2 未来工作展望

（1）建图效率与扩展性优化

当前统一混合图的构建过程仍然依赖多次近邻搜索与候选集合合并，在超大规模数据（如亿级以上）场景下，建图成本仍然较高。未来可探索更加高效的近似建图算法，或结合分布式计算框架提升构建效率。

（2）索引压缩与存储优化

统一图结构在融合多权重邻接关系后，节点的平均度数有所增加，带来额外的存储开销。后续研究可结合向量量化（如 PQ）或图结构压缩技术，在保证召回率的前提下降低内存占用。

（3）动态数据更新机制

当前方法主要面向静态数据集构建，对于数据频繁更新的在线场景（如实时推荐系统），仍需进一步研究支持高效插入与删除的动态图维护机制。

（4）多模态与复杂检索场景扩展

本文实验主要针对文本数据进行稠密与稀疏向量的混合检索，未来可扩展至图像、语音等多模态数据，构建统一的跨模态混合检索索引体系，以适应更复杂的应用需求。

（5）与大模型检索增强系统的结合

伴随着检索增强生成（也就是 RAG）系统不断发展，统一混合图索引有望成为高性能检索模块，嵌入到大语言模型系统当中。今后还可以进一步研究它在问答系统、智能助手这类场景里的实际使用效果，还有对应的优化办法。

总体而言，统一图索引给混合检索提供了一条新的技术思路。跟着相关理论和工程技术不断进步，它在大规模信息检索系统里的应用前景也会变得更加广阔。

参考文献

- [1]Malkov Y, Ponomarenko A, Logvinov A, et al. Approximate nearest neighbor algorithm based on navigable small world graphs[J]. Information Systems, 2014, 45:61-68.
- [2]Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE transactions on pattern analysis and machine intelligence, 2018, 42(4): 824-836.
- [3]FU C, XIANG C, WANG C, et al. Fast approximate nearest neighbor search with the navigating spreading-out graph[J/OL]. Proceedings of the VLDB Endowment, 2019, 12(5): 461-474[2026-05-09]. <https://doi.org/10.14778/3303753.3303754>.
- [4]SUBRAMANYA S J, DEVVRIT, KADEKODI R, et al. DiskANN: fast accurate billion-point nearest neighbor search on a single node[C]//Proceedings of the 33rd International Conference on Neural Information Processing Systems. Red Hook, NY: Curran Associates Inc., 2019: 13766-13776.
- [5]ZHONG X, LI H, JIN J, et al. VSAG: an optimized search framework for graph-based approximate nearest neighbor search[J]. Proceedings of the VLDB Endowment, 2025, 18(12): 5017-5030. DOI: 10.14778/3750601.3750624.
- [6]BAJAJ P, CAMPOS D, CRASWELL N, et al. MS MARCO: A human generated machine reading comprehension dataset[EB/OL]. (2016-11-29)[2026-05-09]. <https://arxiv.org/abs/1611.09268>.
- [7]Patel L, Kraft P, Guestrin C, et al. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data[J]. Proceedings of the ACM on Management of Data, 2024, 2(3): 1-27.
- [8]Gao J, Long C. High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations[J]. Proceedings of the ACM on Management of Data, 2023, 1(2): 1-27.
- [9]Gao J, Long C. Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search[J]. Proceedings of the ACM on Management of Data, 2024, 2(3): 1-27.
- [10]Zuo C, Qiao M, Zhou W, et al. SeRF: segment graph for range-filtering approximate nearest neighbor search[J]. Proceedings of the ACM on Management of Data, 2024, 2(1):

1-26.

[11]Cai Y, Shi J, Chen Y, et al. Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search[J]. Proceedings of the ACM on Management of Data, 2024, 2(6): 1-27.

[12]LI Z, LI Y, ZHU Y, et al. All-in-one graph-based indexing for hybrid search on GPUs[EB/OL]. (2025-11-01)[2026-05-09]. <https://arxiv.org/abs/2511.00855>.

[13]LI R, ZHONG X, JIN J, et al. SINDI: an efficient index for approximate maximum inner product search on sparse vectors[EB/OL]. (2025-09-08)[2026-05-09]. <https://arxiv.org/abs/2509.08395>.

[14]ZHANG H, LIU J, ZHU Z, et al. Efficient and effective retrieval of dense-sparse hybrid vectors using graph-based approximate nearest neighbor search[EB/OL]. (2024-10-23)[2026-05-09]. <https://arxiv.org/abs/2410.20381>.

[15]DONG Y, INDYK P, RAZENSHTEYN I, et al. Learning space partitions for nearest neighbor search[EB/OL]. (2019-01-24)[2026-05-09]. <https://arxiv.org/abs/1901.08544>.

[16]Gao J, Gou Y, Xu Y, et al. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search[J]. Proceedings of the ACM on Management of Data, 2025, 3(3): 1-26.

[17]GAO Y, XIONG Y, GAO X, et al. Retrieval-augmented generation for large language models: a survey[EB/OL]. (2023-12-17)[2026-05-09]. <https://arxiv.org/abs/2312.10997>.

[18]GUO R, LUAN X, XIANG L, et al. Manu: a cloud native vector database management system[J]. Proceedings of the VLDB Endowment, 2022, 15(12): 3548-3561.

[19]INDYK P, MOTWANI R. Approximate nearest neighbors: towards removing the curse of dimensionality[C]//Proceedings of the thirtieth annual ACM symposium on Theory of computing. New York: Association for Computing Machinery, 1998: 604-613. DOI: 10.1145/276698.276876.

[20]LI W, ZHANG Y, SUN Y, et al. Approximate nearest neighbor search on high dimensional data - experiments, analyses, and improvement[J]. IEEE Transactions on Knowledge and Data Engineering, 2020, 32(8): 1475-1488.

[21]PENG Y, CHOI B, CHAN T N, et al. Efficient approximate nearest neighbor search in multi-dimensional databases[J]. Proc. ACM Manag. Data, 2023, 1(1): 54.

[22]ZHENG B, ZHAO X, WENG L, et al. PM-LSH: a fast and accurate LSH framework for high-dimensional approximate NN search[J]. Proc. VLDB Endow., 2020, 13(5):

643-655.

[23]YANG M, LI W, WANG W. Fast high-dimensional approximate nearest neighbor search with efficient index time and space[EB/OL]. (2024-11-06)[2026-05-09].

<https://arxiv.org/abs/2411.06158>.

[24]SUN Y, WANG W, QIN J, et al. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index[J]. Proc. VLDB Endow., 2014, 8(1): 1-12.

[25]VANDERWIEL S P, LILJA D J. Data prefetch mechanisms[J]. ACM Computing Surveys (CSUR), 2000, 32(2): 174-199.

[26]ARORA A, SINHA S, KUMAR P, et al. Hd-index: pushing the scalability-accuracy boundary for approximate knn search in high-dimensional spaces[J]. Proceedings of the VLDB Endowment, 2018, 11(8): 906-919.

[27]FU C, WANG C, CAI D. High dimensional similarity search with satellite system graph: efficiency, scalability, and unindexed query compatibility[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 44(8): 4139-4150.

[28]LU K, KUDO M, XIAO C, et al. HVS: hierarchical graph structure based on Voronoi diagrams for solving approximate nearest neighbor search[J]. Proc. VLDB Endow., 2021, 15(2): 246-258.

[29]JI K, LIU G, DAI N, et al. Enhancing multi-step reasoning abilities of language models through direct Q-function optimization[EB/OL]. (2024-10-11)[2026-05-09]. <https://arxiv.org/abs/2410.09302>.

[30]GOLLAPUDI S, KARIA N, SIVASHANKAR V, et al. Filtered-DiskANN: graph algorithms for approximate nearest neighbor search with filters[C]//Proceedings of the ACM Web Conference 2023. New York: Association for Computing Machinery, 2023: 3406-3416. DOI: 10.1145/3543507.3583552.

谢辞

时光荏苒，光阴如梭，转眼间四年的本科生活即将画上句号。我想，这篇论文的完成，或许就是最圆满的句号了。它像晶莹饱满的晨露，越过长夜而来，蕴含这四年青春时光里我的热血与汗水。这其中，离不开师长与同学的关心与帮助。在此谨向所有给予我支持与帮助的人致以诚挚的感谢。

首先，我要衷心感谢我的指导教师程鹏老师。在论文选题、研究思路以及具体实现过程中，程老师始终给予我耐心细致的指导。从课题方向的确定，到关键技术路线的分析，再到论文结构的反复修改，老师都投入了大量时间与精力。程老师严谨的学术态度和踏实的科研作风，使我受益匪浅，也将对我今后的学习与工作产生深远影响。

其次，感谢实验室的同学和身边的朋友们。在研究论文、建立系统的过程里，和大家聊天讨论经常能给我新的想法，碰到难解决的问题时，也能帮我及时调整方向，让我能一直坚持做下来。有你们的陪伴，这段做科研的日子才变得更充实，也更让人难忘。

同时，衷心感谢学院每一位老师在本科阶段的谆谆教诲。你们精心打造的系统而扎实的专业课程，不光帮我打好了理论基础，碰到复杂的工程问题、算法设计这类难题时，我也能更从容、更有底气，能有这些成长，全离不开老师们一路的指引和鼓励。

此外，感谢我的家人一直以来的理解与支持。在我专注于论文与项目的过程中，家人给予我充分的鼓励与包容，是我最坚强的后盾，最温暖的港湾，使我能够心无旁骛地投身学习科研。

最后，感谢同济大学。谢谢母校给我们提供这么好的校园环境，有丰富的藏书、先进的实验设备，还有好多能开阔眼界的学术讲座，让我不光学到了专业知识，还学会了该怎么自主学习、该怎么独立思考。这四年里，每一个我低头调试代码的深夜，每一次在图书馆窗边看书，突然想通某个问题的瞬间，都有同济的灯火静静陪着我，就是这片土壤，让我能接触到前沿的技术，认识到好多想法一致的朋友，也让我慢慢看清了自己未来要走的方向，这段在同济上学的日子，一定会成为我这辈子都珍藏的底气 and 力量。

谨以此文，向所有关心、帮助和支持过我的人致以最诚挚的感谢。